

Specifikation: Springarens vandring på schackbrädet

Beskrivning

Programmet ska skriva ut en illustration av en schackbräda där det tydligt framgår hur en springare vandrats på brädet utan att komma till en plats mer än en gång. Huvuduppgiften för programmet är att låta användaren mata in en valfri lämplig startruta som existerar på schackbrädet och sedan låta springaren vandra slumpmässigt på schackrutorna tills den inte kan vandra till någon ruta längre.

Användaren ges även olika val på hur hen vill att springaren ska vandra:

- Användaren ges i stället möjligheten att mata ut en egen valfri men lämplig springarvandring.
- Användaren ges i stället möjligheten att mata in en valfri lämplig startruta och programmet undersöker om det finns en springarvandring som passerar varje ruta på schackbrädet en gång. Om det finns en sådan vandring så utför den vandringen.

Programmet presenteras genom ett separat grafikfönster där användaren får se springaren vandra ett steg i taget i enlighet med användarens inmatning.

Algoritm

1. Programmet läser in en inmatning från användaren.
 2. Programmet skriver ut schackbrädan med en slumpvis vandrad springare.
-
1. Programmet läser in flera inmatningar från användaren som föreställer en egen springarvandring.
 2. Programmet skriver ut schackbrädet med en vandrad springare enligt inmatningen.
-
1. Programmet läser in en inmatning från användaren.
 2. Programmet undersöker om det finns en springarvandring som passerar varje ruta en gång utgåendes från den inmatade startrutan.
 3. Programmet skriver ut schackbrädet med en vandrad springare på alla 64 rutor.

Datastrukturer

Schackbrädet representeras av ett Board-objekt. En nästlad lista (matris) på schackbrädets rader och kolumner finns lagrade i en JSON-fil. Listan innehåller även två ytterligare rader och kolumner som agerar som en flyttbuffert åt springaren. En konvention används för att markera rutornas namn och tillstånd på schackbrädet så att programmet vet var springaren inte

får gå. En hjälp lista i form av dictionary för själva schackrutorna finns lagrade i en annan JSON-fil som används för att hantera användarens input. Springarens vandring efter att ha gått ett steg lagras in i ytterligare en lista som agerar som minne åt programmet att inte besöka samma ruta en gång till.

Klasser och metoder

```
class Board:
```

```
    """
```

Attributes:

tile: List of chessboard tiles of the type dict

board_tile: Nestled list (matrix) of the entire chessboard, frame and tiles, of the type list with elements of the type dict

read_data_from_file: Used to generate data from file

knight_position: knight's current position on the board

move: knight's move in accordance with the chess rules

horizontal_tiles: name of the tiles in the horizontal position

vertical_tiles: name of the tiles in the vertical position

position_list: list of knight's positions on the board it's allowed to take

correct_position_list: list of knight's possible positions on the board it can take

position_counter: counter for the amount of steps the knight takes

user_list: list of the user's custom knight walk

tour: knight moving through all 64 tiles on the chessboard once

```
    """
```

```
def __init__(self, tile, board_tile, read_data_from_file):
```

```
    """
```

:param *tile: List of chessboard tiles of the type dict*

:param *board_tile: Nestled list (matrix) of the entire chessboard, frame and tiles, of the type list with elements of the type dict*

:param *read_data_from_file: Used to get data from file*

```
    """
```

```
def get_tile_dict_from_file(self):
```

''''''

Gets the dictionary of the chessboard's tiles from file

:return: *the dictionary containing the chessboard's tiles*

''''''

def update_tile_dict (self):

''''''

Updates the dictionary of the chessboard's tiles

:return: *the updated dictionary containing updated values of the keys in the chessboard's tiles*

''''''

def get_board_list_from_file (self):

''''''

Gets the nested list of the entire chessboard from file

:return: *the nested list containing the entire chessboard*

''''''

def update_board_list (self):

''''''

Updates the nested list of the entire chessboard

:return: *the updated nested list containing updated values of the keys in the entire chessboard*

''''''

def valid_move (self):

''''''

Generates a list of all valid positions and randomizes it

:return: *A valid randomized position*

''''''

def random_position (self, integer):

''''''

Randomizes a given integer from valid_move method

:return: *A randomized value*

''''''

def knight_journey (self):

''''''

Recursive method. Moves the knight until there are no more valid positions, then prints the board.

:return: *(nothing)*

''''''

def knight_walk (self):

''''''

Recursive method. Compares an element from the user's list, starting at the second element, with each element in the valid_position list. If all the elements in the list are valid, the knight moves and the board is printed. If any element is wrong the program asks the user to input correct elements.

:return: *(nothing)*

''''''

def knight_tour (self):

''''''

Analyzes a knight's move two steps ahead to find a tile that have the fewest onward moves and moves the knight one step and repeats the process until it runs out of moves. If the knight visits the remaining 63 tiles after the user's starting position input, the board is printed. If the program doesn't find the Knight's Tour, it informs the user about it.

:return: *(nothing)*

''''''

def board_print (self):

''''''

Prints the entire board with the knight's moves.

:return: *(nothing)*

''''''

Funktioner

```
def check_input (input):
```

```
    """
```

Checks if the user's inputs are of the right format. The expected input is a string corresponding a chess tile. Also checks if the user gives wrong inputs too many times.

:return: *(nothing)*

```
    """
```

```
def check_multiple_inputs (input):
```

```
    """
```

Checks if the user's inputs are of the right format. The expected input is a coma-separated list of strings corresponding the chess tiles. Also checks if the user gives wrong inputs too many times.

:return: *(nothing)*

```
    """
```